

Undirected Graphical Models

Faculty of Information Technology
Monash University, Australia

Week 9

Assessment 2: Structure and Marks

- **Due 11:55 pm, Tuesday 20 October 2026.** In-class interview in Week 12 (mandatory; absence without special consideration = 0).
- **Part I (10 marks).** Linear-chain CRF sleep stager plus at least one baseline that does not model transitions; subject-independent CV (Group K-Fold by Subject ID); Macro F1 as mean $\pm$ s.d.; learned vs empirical transition matrix; hypnograms. Kaggle is 5 of the 10, against fixed thresholds (≥ 0.780 for 5).
- **Part II (5 marks).** Discrete Bayesian network over the CRF's outputs: at least six variables, two of the three basic subgraphs, four queries (marginal, conditional, MAP; one exact, one sampled).
- **Part III (bonus, up to 8).** Web app built with an agentic coding tool that calls your CRF or network; rules files, SPEC.md, tests and a 2-page product brief. Total capped at 25.
- **Part IV (10 marks).** Interview on Parts I to III. A component you cannot explain earns nothing.

Assessment 2: Deliverables

- **The notebook is the report.** Eleven required headings (template on Moodle); markdown limited to 3,000 words; every figure discussed in the cell after it.
- **Three separate Moodle uploads:** notebook PDF, implementation ZIP (notebook, module, saved CRF and network, cover sheet, AI declaration, chat history), brief PDF (Part III only). One combined archive costs 3 marks. Select your final Kaggle submission.
- **Public leaderboard = smoke test.** 8 of 28 test subjects, about ± 0.035 Macro F1. Decide from your own 50-subject CV.
- **This week, Section II:** the same emission model decoded with and without a CRF, and the learned transition matrix read next to the empirical one. That is the Part I comparison in miniature.
- **This week, Section III:** rules, specification, tests, plan, build, review. That is the Part III cycle with a different model.
- **Generative AI** is permitted: declare it (final notebook section and ai_use.txt) and be able to explain every line in Part I and Part II. For Parts I and II, do not generate the submission from the specification directly, work it yourself and use AI to check, question-answering and review. The interview tests your understanding, not the tool's.

Outline



- Markov Networks.
- Independencies in Markov Networks.
- Factor Graphs.
- Conditional Random Fields.
- Agentic Programming for Web Applications.

Learning Outcomes

- After learning this module, you should be able to:
 - Apply undirected graphical models to real-world problems.
 - Understand the connections between undirected graphs and distributions.
 - Analyse independencies in Markov Networks.
 - Train a linear-chain CRF with pytorch-crf and interpret its transition matrix.
 - Follow the specify, test, build, review cycle that turns a fitted model into a web application with an agentic coding tool.

- Alignment with unit learning outcomes:
 - Evaluate the limitations, appropriateness and benefits of data analytics methods for given tasks.
 - Design solutions to real-world problems with data analytic techniques.
 - Assess the results of an analysis.

Misconception

Four students who get together in pairs to work on the homework for a class. For various reasons, only the following pairs meet: Alice and Bob; Bob and Charles; Charles and Debbie; and Debbie and Alice. (Alice and Charles just can't stand each other, and Bob and Debbie had a relationship that ended badly.)

The professor made a mistake in class. Any of the four students may figure out the problem and transmit this understanding to his or her study partners. We need to determine whether a student has the misconception or not.

Misconception – Random Variables

Four students who get together in pairs to work on the homework for a class. For various reasons, only the following pairs meet: Alice and Bob; Bob and Charles; Charles and Debbie; and Debbie and Alice. (Alice and Charles just can't stand each other, and Bob and Debbie had a relationship that ended badly.)

The professor made a mistake in class. Any of the four students may figure out the problem and transmit this understanding to his or her study partners. We need to determine whether a student has the misconception or not.

A: Alice has a misconception or not. $A = 1$ or $A = 0$.

B: Bob has a misconception or not. $B = 1$ or $B = 0$.

C: Charles has a misconception or not. $C = 1$ or $C = 0$.

D: Debbie has a misconception or not. $D = 1$ or $D = 0$.

Misconception – Independence

Four students who get together in pairs to work on the homework for a class. For various reasons, only the following pairs meet: Alice and Bob; Bob and Charles; Charles and Debbie; and Debbie and Alice. (Alice and Charles just can't stand each other, and Bob and Debbie had a relationship that ended badly.)

The professor made a mistake in class. Any of the four students may figure out the problem and transmit this understanding to his or her study partners. We need to determine whether a student has the misconception or not.

Alice and Charlie can't stand each other. $A \perp C \mid \{B, D\}$

Bob and Debbie had a relationship that ended badly. $B \perp D \mid \{A, C\}$

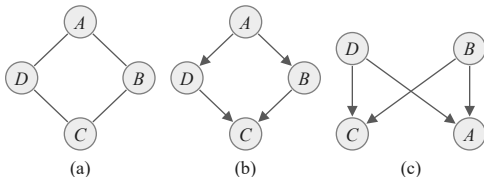
Misconception – Why Not Bayesian Networks

Four students who get together in pairs to work on the homework for a class. For various reasons, only the following pairs meet: Alice and Bob; Bob and Charles; Charles and Debbie; and Debbie and Alice. (Alice and Charles just can't stand each other, and Bob and Debbie had a relationship that ended badly.)

The professor made a mistake in class. Any of the four students may figure out the problem and transmit this understanding to his or her study partners. We need to determine whether a student has the misconception or not.

A Bayesian network cannot satisfy $A \perp C \mid \{B, D\}$, $B \perp D \mid \{A, C\}$.

- A BN would have necessarily extraneous edges.
- An edge in a BN needs to be directional, but the interactions between variables are symmetrical.



Markov Networks

- Markov networks: nodes represent the variables, and edges correspond to direct probabilistic interactions.
- Factor ϕ : a function mapping a set of random variables to real numbers. The set of variables is the scope of the factor.

 $\phi_1(A, B)$
 $\phi_2(B, C)$
 $\phi_3(C, D)$
 $\phi_4(D, A)$

a^0	b^0	30
a^0	b^1	5
a^1	b^0	1
a^1	b^1	10

b^0	c^0	100
b^0	c^1	1
b^1	c^0	1
b^1	c^1	100

c^0	d^0	1
c^0	d^1	100
c^1	d^0	100
c^1	d^1	1

d^0	a^0	100
d^0	a^1	1
d^1	a^0	1
d^1	a^1	100

(a)

(b)

(c)

(d)

Distribution

$$\triangleright P(a, b, c, d) = \frac{1}{Z} \phi_1(a, b) \phi_2(b, c) \phi_3(c, d) \phi_4(d, a)$$

where

$$Z = \sum_{a,b,c,d} \phi_1(a, b) \phi_2(b, c) \phi_3(c, d) \phi_4(d, a)$$

Partition Function

$$\phi_1(a^0, b^1) \phi_2(b^1, c^0) \phi_3(c^0, d^1) \phi_4(d^1, a^0) = 5 * 1 * 100 * 1 = 500$$

$\phi_1(A, B)$			$\phi_2(B, C)$			$\phi_3(C, D)$			$\phi_4(D, A)$		
a^0	b^0	30	b^0	c^0	100	c^0	d^0	1	d^0	a^0	100
a^0	b^1	5	b^0	c^1	1	c^0	d^1	100	d^0	a^1	1
a^1	b^0	1	b^1	c^0	1	c^1	d^0	100	d^1	a^0	1
a^1	b^1	10	b^1	c^1	100	c^1	d^1	1	d^1	a^1	100
(a)			(b)			(c)			(d)		

Gibbs Distributions

- Def: A distribution P_{Φ} is a Gibbs distribution $\Phi = \{\phi_1(D_1), \dots, \phi_K(D_K)\}$ parameterized by a set of factors if it is defined as follows:

$$P(X_1, \dots, X_n) = \frac{1}{Z} \widetilde{P}_{\Phi}(X_1, \dots, X_n)$$

where

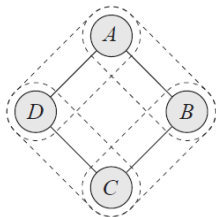
$$\widetilde{P}_{\Phi}(X_1, \dots, X_n) = \phi_1(D_1) \times \phi_2(D_2) \times \dots \times \phi_K(D_K)$$

$$Z = \sum_{X_1, \dots, X_n} \widetilde{P}_{\Phi}(X_1, \dots, X_n)$$

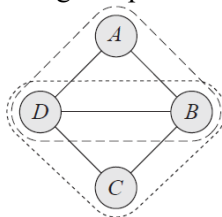
Partition Function

Factorization

- **Markov network factorization:** a distribution P_{Φ} with $\Phi = \{\phi_1(D_1), \dots, \phi_K(D_K)\}$ factorizes over a Markov network H if each D_k ($k = 1, \dots, K$) is a *complete subgraph (Clique)* of H .
- **Maximal clique:** a complete subgraph that cannot be enlarged by adding another vertex without breaking completeness.



(a)
 $\{A, B\}$
 $\{B, C\}$
 $\{C, D\}$
 $\{D, A\}$



(b)
 $\{A, B, D\}$
 $\{B, C, D\}$

Reduced Markov Networks

- Conditional distribution $P(A, B \mid C = 1)$.

a^1	b^1	c^1	$0.5 \cdot 0.5 = 0.25$
a^1	b^1	c^2	$0.5 \cdot 0.7 = 0.35$
a^1	b^2	c^1	$0.8 \cdot 0.1 = 0.08$
a^1	b^2	c^2	$0.8 \cdot 0.2 = 0.16$
a^2	b^1	c^1	$0.1 \cdot 0.5 = 0.05$
a^2	b^1	c^2	$0.1 \cdot 0.7 = 0.07$
a^2	b^2	c^1	$0 \cdot 0.1 = 0$
a^2	b^2	c^2	$0 \cdot 0.2 = 0$
a^3	b^1	c^1	$0.3 \cdot 0.5 = 0.15$
a^3	b^1	c^2	$0.3 \cdot 0.7 = 0.21$
a^3	b^2	c^1	$0.9 \cdot 0.1 = 0.09$
a^3	b^2	c^2	$0.9 \cdot 0.2 = 0.18$



a^1	b^1	c^1	0.25
a^1	b^2	c^1	0.08
a^2	b^1	c^1	0.05
a^2	b^2	c^1	0
a^3	b^1	c^1	0.15
a^3	b^2	c^1	0.09

Outline

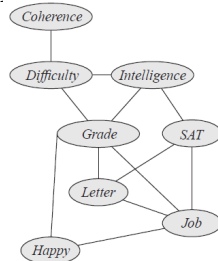


- Markov Networks.
- Independencies in Markov Networks.
- Factor Graphs.
- Conditional Random Fields.
- Agentic Programming for Web Applications.

Independencies

- Active path: Let H be a Markov network structure, and $X_1 - \dots - X_k$ be a path in H . Let $Z \subseteq X$ be a set of observed variables, the path $X_1 - \dots - X_k$ is active given Z if none of the X_i is in Z .
- $\mathbf{Z}$ separates $\mathbf{X}$ and $\mathbf{Y}$ in H , denoted by $\text{sep}_H(\mathbf{X}; \mathbf{Y} | \mathbf{Z})$, if there is no *active* path between any node $X_i \in \mathbf{X}$ and $Y_i \in \mathbf{Y}$, given $\mathbf{Z}$.
- Global independencies:

$$I(H) = \{(\mathbf{X} \perp \mathbf{Y} | \mathbf{Z}) : \text{sep}_H(\mathbf{X}; \mathbf{Y} | \mathbf{Z})\}$$



Local Markov Assumptions

- Pairwise independencies: Let H be a Markov network over V ,

$$I_p(H) = \{(X \perp Y \mid V/\{X, Y\}) : X - Y \notin H\}$$

* X and Y are independent given the remaining nodes, if there is no edge between X and Y .

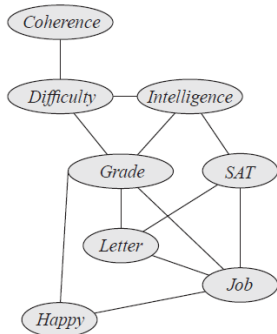
- Local dependencies (Markov blanket): Let H be a Markov network over V and $MB_H(X)$ be the neighbours of X in H ,

$$I_l(H) = \{(X \perp V/\{X, MB_H(X)\} \mid MB_H(X)) : X \in V\}$$

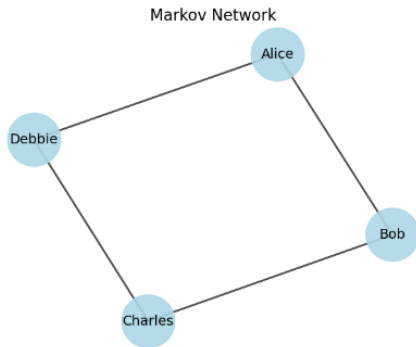
* X is independent of the rest of the nodes in the graph given its immediate neighbours.

Relationship between Independencies

- The following statements are equivalent for a positive distribution P .
 - P satisfies global independencies $I(H)$.
 - P satisfies pairwise independencies $I_p(H)$.
 - P satisfies local independencies $I_l(H)$.
- A distribution is positive, if $P(x) > 0$ for all x .



Activity I: Independence & Inference



Compute $P(\text{Alice} \mid \text{Bob} = 1, \text{Debbie} = 1, \text{Charles} = 1)$ and $P(\text{Alice} \mid \text{Bob} = 1, \text{Debbie} = 1, \text{Charles} = 0)$ and $P(\text{Alice} \mid \text{Bob} = 1, \text{Debbie} = 1)$. What's your conclusion?

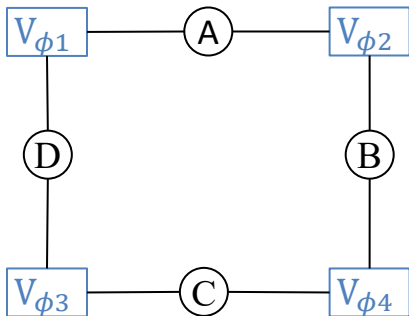
Outline



- Markov Networks.
- Independencies in Markov Networks.
- **Factor Graphs.**
- Conditional Random Fields.
- Agentic Programming for Web Applications.

Factor Graph

- Def: A factor graph is an undirected graph containing two types of nodes: variable nodes and factor nodes. The graph only contains edges between variable nodes and factor nodes.



$$P(A, B, C, D) = \frac{1}{Z} \prod_i \phi_i$$

Log-Linear Models

- Rewrite each factor using an energy function.
 - $\phi_1(A, B) = \exp(-\sum_{i \in \{0,1\}} \sum_{j \in \{0,1\}} w_{ij} I(A = i, B = j)).$
 - $\phi_2(B, C) = \exp(-\sum_{i \in \{0,1\}} \sum_{j \in \{0,1\}} w_{ij} I(B = i, C = j)).$
 - ...
- $\phi(D) = \exp(-\epsilon(D))$, where $\epsilon(D)$ is an *energy* function.

$$P(A, B, C, D) = \frac{1}{Z} \prod_i \exp(-\epsilon_i(D_i))$$

$\epsilon_1(A, B)$			$\epsilon_2(B, C)$			$\epsilon_3(C, D)$			$\epsilon_4(D, A)$		
a^0	b^0	-3.4	b^0	c^0	-4.61	c^0	d^0	0	d^0	a^0	-4.61
a^0	b^1	-1.61	b^0	c^1	0	c^0	d^1	-4.61	d^0	a^1	0
a^1	b^0	0	b^1	c^0	0	c^1	d^0	-4.61	d^1	a^0	0
a^1	b^1	-2.3	b^1	c^1	-4.61	c^1	d^1	0	d^1	a^1	-4.61
(a)			(b)			(c)			(d)		

Log-Linear Models

- A distribution P is a log-linear model over a Markov network H if it is associated with:
- A set of features $F = \{f_1(D_1), \dots, f_k(D_k)\}$, where each D_i is a complete subgraph of H .
 - A set of weights $w_1, \dots, w_k$, such that

$$P(X_1, \dots, X_n) = \frac{1}{Z} \exp \left[- \sum_{i=1}^k w_i f_i(D_i) \right]$$

Where a feature $f_i(D_i)$ maps values of D_i to real values.

Outline



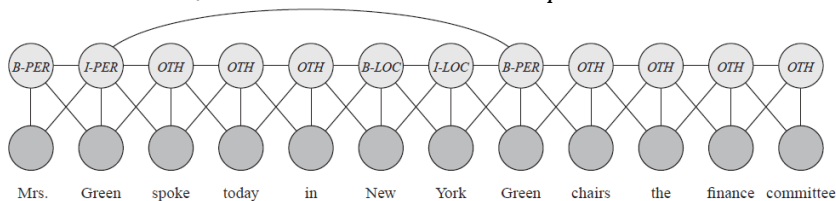
- Markov Networks.
- Independencies in Markov Networks.
- Factor Graphs.
- Conditional Random Fields.
- Agentic Programming for Web Applications.

Conditional Random Fields

- A conditional random field (CRF) is an undirected graph H whose nodes correspond to $X \cup Y$, the network is annotated with a set of factors $\phi_1(D_1), \dots, \phi_m(D_m)$ such that each $D_i \not\subseteq X$. The network encodes a conditional distribution as follows:

$$P(Y|X) = \frac{1}{Z(X)} \tilde{P}(Y, X)$$

$$\tilde{P}(Y, X) = \prod_{i=1}^m \phi_i(D_i) \quad Z(X) = \sum_Y \tilde{P}(Y, X)$$



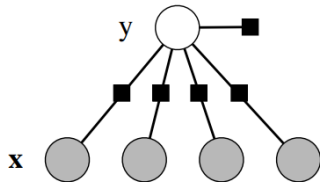
Revisit Logistic Regression

➤ Define

- a pairwise potential: $\phi_i(X_i, Y) = \exp\{w_i X_i I(Y = 1)\}$.
- a single node potential: $\phi_0(Y) = \exp\{w_0 I(Y = 1)\}$.

➤ We have

- $\tilde{P}(Y = 1 | x_1, \dots, x_m) = \exp\{w_0 + \sum_{i=1}^m w_i x_i\}$.
- $\tilde{P}(Y = 0 | x_1, \dots, x_m) = \exp\{0\} = 1$.
- $P(Y = 1 | x_1, \dots, x_m) = \frac{\tilde{P}(Y=1|x_1, \dots, x_m)}{\tilde{P}(Y=1|x_1, \dots, x_m) + \tilde{P}(Y=0 | x_1, \dots, x_m)}$.
- $P(Y = 1 | x_1, \dots, x_m) = \sigma(w_0 + \sum_{i=1}^m w_i x_i)$
 where $\sigma(u) = \frac{\exp\{u\}}{1 + \exp\{u\}}$.



Linear-Chain CRF

➤ Named entity recognition.

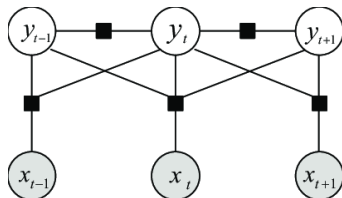
On the 15th of September **DATE** , Tim Cook **PERSON** announced that
Apple **ORG** wants to acquire ABC Group **ORG** from New York **GPE**
for 1 billion dollars **MONEY**

➤ Part-of-Speech tagging.

Visualizing **VERB** Part-of-Speech **NOUN** Tags **NOUN**
with **ADP** NLTK **NOUN** and **CONJ** Spacy **NOUN**

Linear-Chain CRF

- Factor graph representation.



$$\begin{aligned}
 \text{➤ } P(Y_1, \dots, Y_l | x_1, \dots, x_l) &= \frac{P(Y_1, \dots, Y_l, x_1, \dots, x_l)}{\sum_{y_1, \dots, y_l} P(Y_1 = y_1, \dots, Y_l = y_l, x_1, \dots, x_l)} \\
 &= \frac{1}{Z} \prod_{t=1}^l \exp\{\sum_{k=1}^K w_k f_k(y_t, y_{t-1}, x_t)\} \\
 &= \frac{\prod_{t=1}^l \exp\{\sum_{k=1}^K w_k f_k(y_t, y_{t-1}, x_t)\}}{\sum_{y'_1, \dots, y'_l} \prod_{t=1}^l \exp\{\sum_{k=1}^K w_k f_k(y'_t, y'_{t-1}, x_t)\}}
 \end{aligned}$$

Feature Design for NER

- $f_1(y_t, y_{t-1}, x_t) = f_1(y_t, x_t) = I(Y_t = B - ORG, X_t = \text{"Apple"})$.
- $f_2(y_t, y_{t-1}, x_t) = f_2(y_t, y_{t-1}) = I(Y_t = I - ORG, Y_{t-1} = B - ORG)$.
- $f_3(y_t, x_t) = I(Y_t = B - ORG, X_t \text{ is uppercase})$.
- ...

Model Training

- Training objective: Given a data set of N examples, the log-likelihood is defined as:

$$\min_w - \sum_{i=1}^N \log P(\mathbf{y}_i | \mathbf{x}_i)$$

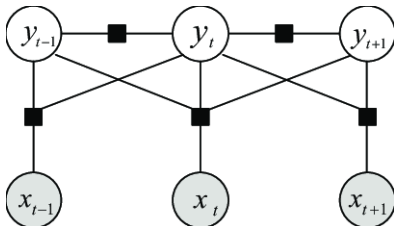
- For $(\mathbf{y}_i, \mathbf{x}_i)$, we provide a set of features $F = \{f_i(D_i)\}_{i=1}^k$, where $f_i(D_i)$ is a feature function defined over the variables in D_i .

$$p(\mathbf{y}_i | \mathbf{x}_i) = \frac{1}{Z(\mathbf{x}_i)} \prod_{t=1}^l \exp\{\sum_{k=1}^K w_k f_k(y_t, y_{t-1}, x_t)\}$$

$$\text{Where } Z(x) = \sum_{y'_1, \dots, y'_l} \prod_{t=1}^l \exp\{\sum_{k=1}^K w_k f_k(y'_t, y'_{t-1}, x_t)\}.$$

More on Model Training

- The log-likelihood function has no local optima.
- There is no analytical form for its maximum. We resort to iterative methods, e.g. L-BFGS, to find model parameters.
- Inference is required for CRFs.
- Feature functions can be learned by using neural networks.
- This week: emissions from a network (Embedding $\rightarrow$ Linear), transitions from a torchcrf.CRF layer, trained jointly with Adam.



Activity II: Reading the CRF

- Compare the per-token baseline with the CRF on the two validation sentences and on the per-tag F1 of the test set. Which tags gained most from the transition layer, and why did the B- tags gain less?
- Read the learned transition matrix next to the empirical one counted from the training tags. Which transitions are favoured, which are suppressed, and are any of the suppressed ones impossible under BIO?
- Where do the two matrices disagree? Explain the disagreement from the fact that transition scores are added to emission scores.

Outline

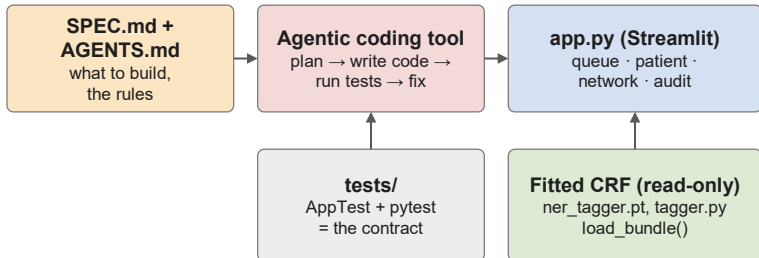


- Markov Networks.
- Independencies in Markov Networks.
- Factor Graphs.
- Conditional Random Fields.
- Agentic Programming for Web Applications.

From a Fitted Model to a Product

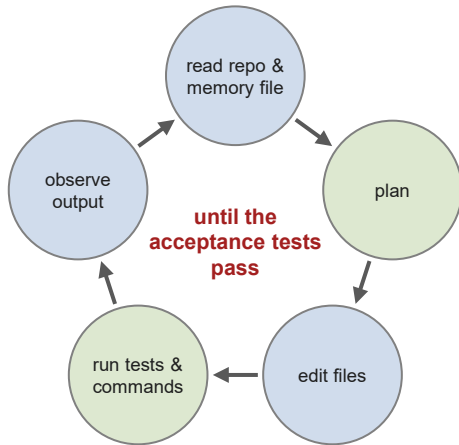
- You have a CRF that turns a sentence into entity tags for a clinic.
- A clinic cannot use a notebook. It needs a queue, a patient view, a network view and an audit trail.
- An agentic coding tool takes a written specification, a rules file and a model file, produces the application, and runs the tests you wrote.
- **The skill being taught is not prompting. It is specifying, verifying and reviewing.**

The agent builds the app around a fixed model and a fixed contract



What an Agentic Coding Tool Does

- It is not autocomplete. It runs a loop in your project folder:
 - reads the code and a rules file
 - plans, edits several files, runs shell commands and tests
 - reads the output and repeats until the task is done
- You control what it may do without asking: read-only, edit-only, or full auto inside a sandbox.
- Rules file: Markdown the agent reads on every start; stack, pinned versions, what never to touch, what “done” means.
 - AGENTS.md: read by all three tools. CLAUDE.md, GEMINI.md: tool-specific.
- Headless mode (one prompt, no chat) lets you script it once the workflow is stable.



Three Tools, One Idea

	Claude Code (Anthropic)	Codex (OpenAI)	Antigravity (Google)
Install	<code>curl -fsSL https://claude.ai/install.sh bash</code>	<code>npm i -g @openai/codex</code>	IDE and 2.0 app from antigravity.google.com/download ; CLI: <code>curl -fsSL https://antigravity.google.com/cli/install.sh bash</code>
Sign in	Claude Pro/Max/Team or Console API key	ChatGPT account or API key	Google account; personal accounts get AI credits; enterprise via Gemini Enterprise
Rules file	CLAUDE.md (also reads AGENTS.md); /init writes one	AGENTS.md (global ~/.codex, repo, folder)	AGENTS.md or GEMINI.md; .agents/rules/*.md; global ~/.gemini/
Autonomy	permission prompts; plan mode; hooks	--ask-for-approval; OS sandbox; --full-auto	permission presets; terminal sandbox; Plan Review Policy; hooks
Extras	MCP, skills, subagents	MCP, skills, cloud agent (Codex Web)	IDE + Agent Manager + CLI on one engine; artefacts (plan, walkthrough); browser agent; subagents; MCP; skills

Antigravity IDE: Anatomy

- **Three surfaces, one agent engine.** Antigravity IDE (VS Code-based editor with the agent built in, v2.5.5), Antigravity 2.0 (standalone agent manager: projects, worktrees, parallel agents, no editor; v2.18.1, 28 Sep 2026), and the agy CLI (v1.2.11). Rules, artefacts, permissions and skills are shared.
- **In the IDE:** Editor with Tab completion; Agent side panel for the conversation; Agent Manager to run several agents in parallel; Review Changes pane for the diff; integrated terminal and Git.
- **Plan before code.** Type `/plan` and the agent drafts an Implementation Plan artefact you read and edit before any file changes. The Plan Review Policy setting decides whether every plan waits for you.
- **Artefacts** are how the agent reports: Implementation Plan, task list, Walkthrough, screenshots and browser recordings. Read them; they are the agent's claims, and your tests are the check.
- **Rules and Skills.** AGENTS.md or GEMINI.md in the project root (always on); `.agents/rules/*.md` with a trigger; global rules in `~/.gemini/`. Multi-step procedures go in Skills (Workflows are migrating to Skills).
- **Permissions and sandbox.** Presets from review-everything to agent-driven; terminal commands run in a sandbox and ask before leaving it; `.env`, `.git` and `tests/` can be declared off-limits.
- **Browser agent** opens the app you built, clicks through it and records the session; useful, but not a substitute for the tests.

Source: antigravity.google/docs, checked 1 October 2026.

A Workflow That Works

- **1. Write the spec first.** Files, inputs, outputs and acceptance tests in plain language (SPEC.md).
- **2. Write the rules file (AGENTS.md).** Stack, pinned versions, the data contract (load_bundle), what the agent must never edit (model file, tests, data, labels), what “done” means.
- **3. Ask for a plan before code.** /plan in Antigravity, plan mode in Claude Code. Read it. Fix the design there; it is ten times cheaper than fixing code.
- **4. One layer at a time.** tagger.py with its tests, then app.py. Tests before application code.
- **5. Make the agent run the tests.** “Done” means the suite passes, not that the code was written.
- **6. Review the diff, then commit.** Git is the undo button. Small commits, one intent each.
- **7. Iterate with evidence.** Paste the failing test or the traceback, not “it doesn't work”.

Writing the Specification

- SPEC.md for the Week 9 app (Section III notebook, step 3), abridged:
Build `app.py`, a Streamlit app for the CRF in `ner_tagger.pt`, loaded once with `load_bundle()` from `tagger.py`. A radio chooses the model (CRF or baseline), another the input (an example validation sentence, or a typed sentence split on whitespace). Show the sentence with entities highlighted, a token / predicted / gold table with token accuracy, and a heatmap of the CRF transition matrix.

Acceptance: `python -m pytest -q tests` passes (the tests exist; read them first). No dependency beyond `streamlit`, `pandas`, `matplotlib`, `torch` and `pytorch-crf`. Every function has a docstring; under 80 lines.

- Note what is in it: file names, the data contract (what `load_bundle` returns), the tests, and the boundaries. Note what is not: how to write the code.

Verification: the Tests Are the Contract

- Agents are fluent, fast and sometimes wrong in ways that read as right. The defence is executable.
- **Tests on the model layer:** the bundle loads; one tag per token for both models; every tag is a real tag.
- **UI tests without a browser:** streamlit.testing.v1.AppTest runs the page, asserts a table and a metric exist, changes the selectbox and checks the table changed.
- **Failure modes seen in this unit:**
 - API from the wrong version (pgmpy MarkovNetwork vs DiscreteMarkovNetwork; pomegranate 0.14 vs 1.x): pin versions in the rules file.
 - Silent numerical bugs (zero cells in a CPT, NaN in a Gibbs sweep, an off-by-one in the tag list): assert the invariants in tests.
 - Leakage: an agent will happily split by row, not by Subject ID. Your CV test must check the grouping key.
 - Fabricated results: a number typed into a README that was never computed. Numbers come from test output only.

Guardrails and Academic Integrity

- **Data stays out of reach.** Never give an agent the test labels or write access to data/; no datasets inside app/. Put that line in the rules file.
- **Start restrictive.** Approval prompts on; full-auto only inside a sandbox with the network disabled.
- **Review every diff.** You are the author of record. If you cannot explain a line, delete it or have it explained and re-check against the lecture.
- **Declare tool use.** Which tool, which model, which files: in the notebook's final section and in ai_use.txt. Part III also submits the rules files and SPEC.md in their final state.
- **The interview is the test.** You must explain the whole pipeline, including code an agent wrote. Marks are withheld for any component you cannot explain.
- **What agents are good at:** scaffolding, glue, tests, refactors. What they are not: deciding your grouping key, your metric or your prior.

Demo: From CRF to Streamlit App

- Each step in Week_9_Section_III_From_Model_to_App.ipynb has a markdown cell (what to do, what to check) and a code cell that writes the input to Antigrahy: a prompt under prompts/, or a file the agent reads.
- **1. Package and modularise (agent).** Prompt 1 hands the agent the Section I/II notebook and fixes the interface: data.py, tagger.py with load_bundle() → Bundle (predict, transitions, tags, examples), train.py. Check: F1 0.535 / 0.640 reproduced, no change to architecture or hyperparameters.
- **2. Rules (you).** AGENTS.md: pinned requirements.txt, the Bundle contract, never touch the model files, tests/ or data/, “done” = pytest passes, no uncomputed numbers.
- **3. Spec (you)** SPEC.md. **4. Tests (you):** test_tagger.py passes now, test_app.py fails until app.py exists. Commit the snapshot.
- **5. Plan, then build (agent).** “/plan Read SPEC.md, AGENTS.md and tests/ ... no code yet.” Read the Implementation Plan; approve; “Implement, run pytest, paste the output.” Failures: paste the traceback, not adjectives.
- **6. Review (you).** pytest yourself; Review Changes or git diff (only app.py should change); prompt 6 asks for a line-by-line explanation of the bundle.predict call; delete what you cannot explain.
- **7. Run (you).** streamlit run app.py: African I-MISC vs B-MISC, Air France missed by both, a typed name the model never saw; optional browser-agent prompt.

Learning Resources

- **Claude Code (Anthropic)**
 - Docs: code.claude.com/docs (Quickstart, CLAUDE.md, plan mode, hooks, skills).
 - Courses (free, Anthropic Academy): anthropic.skilljar.com/claude-code-101 and [.../claude-code-in-action](https://anthropic.skilljar.com/claude-code-in-action).
 - Guide: anthropic.com/engineering/claude-code-best-practices.
- **Codex (OpenAI)**
 - Docs: developers.openai.com/codex (Quickstart; guides/agents-md; learn/best-practices).
 - Source and docs/ folder: github.com/openai/codex.
- **Antigravity (Google)**
 - Docs: antigravity.google/docs (Getting Started; Rules; Artifacts; Antigravity IDE overview; CLI tutorial).
 - Changelog: antigravity.google/docs/changelog, new releases roughly weekly.
- **For all three:** the skill is the same. Read AGENTS.md conventions once (agents.md), then practise on a small project with tests you wrote first.

Links checked 1 October 2026.

Summary

- What we learned:
 - Model real-world problems with Markov networks.
 - Analyse independencies in Markov networks.
 - Understand the connections between Markov networks and distributions.
 - Train a linear-chain CRF with pytorch-crf and read its transition matrix.
 - Specify, test, build and review a web application around a model with an agentic coding tool (Antigravity IDE).
 - Factor graphs, log-linear models and CRFs.
- Relevant readings:
 - Sections 4.1 – 4.3.2, 4.4.1, 4.5.1, 4.6.1, 20.3.1 and 20.3.2 of the book “Probabilistic Graphical Models, Principles and Techniques”.
- Acknowledgement
 - Some figures used in these slides are from the book “Probabilistic Graphical Models, Principles and Techniques” and Pgmpy library.